DLDCA Outlab: Verilog Comparators

Week 1

August 6, 2026

Introduction

In this outlab we are going to start with a simple set of problems through which we will understand how to build comparators using basic Verilog modules. The lab consists of two tasks, with the second task showing how a wider comparator can be constructed from a smaller one.

Necessary tools

We shall be using `icarus-verilog` (or) `iverilog` in order to compile verilog HDL (Hardware Description Language). To visualize waveforms we shall use the VS Code extension 'Vaporview'. Before starting the lab, ensure that both tools are installed in your system. This can be verified by running the following:

```
$ iverilog
iverilog: no source files.

Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
                [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
                [-D macro[=defn]] [-I includedir] [-L moduledir]
                [-M [mode=]depfile] [-m module]
                [-N file] [-o filename] [-p flag=value]
                [-s topmodule] [-t target] [-T min|typ|max]
                [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
```

Structure of submission/templates

Submission format:

```
your-roll-no/
|- task1/
    |- comparator.v (TODO)
    |- tb_task1.v
|- task2/
    |- comparator.v
    |- fourbit_comparator.v (TODO)
    |- tb_task2.v
```

The folder structure of the expected submission for this outlab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

Task 1: Comparing Bits

The goal of this task is to create a simple one bit comparator module that takes two input bits (a, b) and produces three outputs: `eq`, which is high when `a` and `b` are equal; `gt`, which is high when `a` is greater than `b`; and `lt`, which is high when `a` is less than `b`.

The truth table for the circuit is as follows

a	b	eq	gt	lt
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

Derive the formulae for the `eq`, `gt`, and `lt` wires from the truth table using karnaugh maps or direct reasoning.

You have been provided a template module file (`comparator.v`), you are expected to fill in the logic to complete the comparator. In order to test your implementation we have also provided you a testbench (`tb_task1.v`).

To run your code, use the makefile:

```
$ make task1
```

This will open a waveform viewer within VSC where you can see the output.

Make sure to go through the testbench code to understand how to output your testbench output to `.vcd` files in order to visualize them with the extension.

Task 2: Scaling Up

Having built a 1-bit comparator in Task 1, we can now use it as a building block to construct a wider comparator. This task asks you to build a 4-bit comparator by chaining together four copies of your 1-bit `comparator` module.

The 4-bit comparator takes two 4-bit inputs `A` and `B`, and produces the same three outputs as Task 1: `eq`, `gt`, and `lt`. The bits are ordered from least significant to most significant, so `A[0]`/`B[0]` are the least significant bits and `A[3]`/`B[3]` are the most significant bits.

The comparison should be performed from the most significant bit to the least significant bit. If the most significant bits are equal, move on to the next bit; otherwise, the first bit position where the inputs differ determines whether `A` is greater than or less than `B`. The `eq` output should only be high when all four bit comparisons are equal.

Your completed `comparator.v` from Task 1 has been copied into this folder, since you will need to instantiate it four times inside `fourbit_comparator.v`. Each instance compares one bit position. You should then combine the outputs so that the final result reflects the most significant differing bit.

A testbench (`tb_task2.v`) has been provided, which drives a handful of directed test cases and prints the inputs and outputs at each step, while also dumping a waveform for you to inspect.

To run your code, use the makefile:

```
$ make task2
```

This will open a waveform viewer within VSC where you can see the output.